

PYTHON BACKEND FOUNDATIONS

Designing Strong Backend Fundamentals

with Python for Scalable Applications

Ishfaq Ahmad Dar

A Practical Guide to Backend Engineering

**Covering Python Core, Async Programming, OOP, Testing, and Interview
Preparation**

First Edition

2026

© 2026 Ishfaq Ahmad Dar

All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without prior permission of the author.

□ About This Book

This book is designed to build a **strong foundation in Python for backend development**. It focuses on practical concepts, interview preparation, and real-world backend skills.

It is intended for:

- Students preparing for backend roles
- Beginners learning Python for backend
- Developers transitioning into backend engineering

□ What You Will Learn

- Python fundamentals for backend
- Functions, decorators, and advanced concepts
- Object-oriented programming
- Asynchronous programming
- Error handling and logging
- Testing and best practices
- Interview-focused problem solving

□ How to Use This Book

- Study **1 chapter per day**
- Practice coding after each chapter

- Revise using cheat sheets
- Build mini projects after each section

□ **Goal of This Book**

To make you a **strong Python backend developer** with:

- Clear fundamentals
- Practical coding skills
- Interview readiness

Start your backend journey with confidence. □

Chapter 1: Python Fundamentals

Data Types, Mutability, Comprehensions & Memory Model

1. Introduction (Why This Matters in Backend)

Backend systems are designed to **process, store, and transfer data efficiently**. Every request that hits your server (API call, database query, authentication check) involves **data structures and memory operations**.

In real backend workflows:

- Client sends JSON → Python converts it into **dict**
- Database returns rows → stored as **list of dicts**
- Caching systems → depend on **immutable keys (tuples, strings)**
- High-traffic systems → require **efficient memory handling**

□ This means:

- Choosing the wrong data type = **slow system**
- Not understanding mutability = **unexpected bugs**
- Ignoring memory behavior = **hard-to-debug issues**

□ Interview Perspective:

- Almost every Python backend interview includes:
 - Data types
 - Mutability
 - Memory model
 - Comprehensions

This chapter builds your **mental model of how Python actually works**.

2. Core Concepts

2.1 Python Data Types (In-Depth Theory + Practice)

Python is a **dynamically typed, high-level language**, meaning:

- You don't declare variable types explicitly
- Types are determined at runtime

Example:

```
x = 10      # int
x = "Ali"   # now string
```

□ Python stores everything as **objects**, not raw values.

□ List (Dynamic Array Structure)

A list is a **resizable array of references to objects**.

Internal Working:

- Python lists do NOT store actual values
- They store **pointers (references)** to objects
- When list grows → Python allocates extra memory (over-allocation strategy)

Example:

```
numbers = [1, 2, 3]
```

Memory concept:

numbers → [ref → 1, ref → 2, ref → 3]

Advanced Properties:

- Supports heterogeneous data

```
data = [1, "Ali", 3.5, True]
```

- Supports slicing

```
print(data[1:3])
```

- Supports negative indexing

```
print(data[-1]) # last element
```

Performance Insight:

- Append → $O(1)$ (amortized)
- Insert at beginning → $O(n)$

Backend Insight:

- Large lists → expensive operations
- Prefer generators for large data

□ **Tuple (Immutable Data Container)**

A tuple is an **immutable sequence**.

Internal Behavior:

- Stored as fixed-size object
- No resizing → memory efficient
- Faster iteration than list

Important Concept: Hashability

Tuples can be used as dictionary keys because:

- They are immutable
- Their hash value does not change

```
location = {(10, 20): "Office"}
```

Nested Tuple:

```
t = (1, [2, 3])
```

- Tuple is immutable, but inner list is mutable
- This is a common interview trap

□ Set (Hash-Based Collection)

A set is implemented using a **hash table**, similar to dictionary keys.

Internal Mechanism:

- Each element $\rightarrow$ hashed
- Stored in bucket
- No order guarantee

Example:

$s = \{1, 2, 3\}$

Important Properties:

- No duplicates
- Fast membership testing

Mathematical Operations:

$a = \{1, 2, 3\}$

$b = \{2, 3, 4\}$

`print(a & b) # intersection`

`print(a | b) # union`

Backend Insight:

- Useful for:
 - Removing duplicates
 - Permission checking
 - Comparing datasets

□ Dictionary (Hash Map)

A dictionary is a **mapping between keys and values**, implemented using hashing.

Internal Mechanism:

1. Key $\rightarrow$ hash function
2. Hash $\rightarrow$ memory index
3. Value stored at that location

Example:

```
user = {"id": 1, "name": "Ali"}
```

Important Properties:

- Keys must be immutable
- Values can be anything

Collision Handling:

- If two keys have same hash → Python resolves internally

Ordered Nature:

- Python 3.7+ → preserves insertion order

Backend Insight:

- Dict is the **most used structure**
- Represents:
 - JSON
 - API payloads
 - Database records

2.2 Mutability vs Immutability (Deep Conceptual Understanding)

☐ What is Mutability?

Mutability means:

- ☐ Whether an object can change **without changing its identity**

Example:

```
lst = [1, 2]
id_before = id(lst)

lst.append(3)

id_after = id(lst)

print(id_before == id_after)  # True
```

□ Same object modified

□ What is Immutability?

```
x = 10
id_before = id(x)

x = x + 5

id_after = id(x)

print(id_before == id_after)  # False
```

□ New object created

□ Why Python Uses Both?

- Mutable → flexibility
- Immutable → safety + performance

❑ String Immutability

```
s = "hello"  
s = s + " world"
```

❑ New string created (old not modified)

❑ Benefits of Immutability:

- Thread-safe
- Can be hashed
- Safe in caching systems

❑ Mutable Pitfalls (Important)

```
a = [[1, 2]] * 3  
a[0][0] = 100  
  
print(a)
```

❑ Output: `[[100, 2], [100, 2], [100, 2]]`

❑ All inner lists share same reference

2.3 List Comprehensions (Advanced Theory)

List comprehensions are based on **functional programming concepts**.

Concept:

Instead of writing loops, we express:

- “What we want” instead of “How to do it”

Syntax Breakdown:

```
[expression for item in iterable if condition]
```

Execution Steps:

1. Iterate over iterable
2. Apply condition
3. Apply transformation
4. Store in new list

Example:

```
nums = [1, 2, 3, 4]
```

```
result = [x*x for x in nums if x % 2 == 0]
```

Performance Insight:

- Faster than loops (optimized internally)
- But avoid overly complex comprehensions

Nested Comprehension (Advanced):

```
matrix = [[1, 2], [3, 4]]
```

```
flat = [num for row in matrix for num in row]
```

Backend Insight:

- Used in:
 - Data cleaning
 - Response formatting
 - Filtering

2.4 Dictionary Comprehension (Deep Understanding)

Concept:

Used to create dictionaries dynamically

Syntax:

```
{key: value for item in iterable if condition}
```

Example:

```
nums = [1, 2, 3]
```

```
square_map = {x: x*x for x in nums}
```

Advanced Use:

```
users = ["Ali", "Sara"]
```

```
user_dict = {name: len(name) for name in users}
```

Backend Use:

- Mapping IDs to objects
- Creating lookup tables
- Transforming API data

2.5 Python Memory Model (Very Important Theory)

□ Everything is an Object

In Python:

- Numbers → objects
- Strings → objects
- Lists → objects

☐ **Names vs Objects**

```
a = 10
```

- ☐ a is NOT the value
- ☐ a is a reference (label) pointing to object 10

☐ **Multiple References**

```
a = [1, 2]  
b = a
```

- ☐ Both point to same object

☐ **Object Identity**

```
print(id(a))
```

- ☐ Returns memory address

☐ **Garbage Collection**

Python automatically deletes objects when:

- No references remain

□ Reference Counting

Each object tracks how many references point to it.

□ Function Argument Passing

Python uses:

□ Pass-by-object-reference

```
def update(x):  
    x.append(10)
```

```
data = [1, 2]  
update(data)
```

□ Original list changes

□ Immutable Case:

```
def update(x):  
    x = x + 10
```

```
a = 5  
update(a)
```

```
print(a)  # unchanged
```

□ Backend Insight:

- Mutable objects → can cause side effects

- Immutable → safer for APIs and caching

3. Real-World Examples

API Transformation

```
users = [{"name": "Ali"}, {"name": "Sara"}]  
names = [u["name"] for u in users]
```

Deduplication

```
ids = [1, 2, 2, 3]  
unique = list(set(ids))
```

Lookup Optimization

```
users = {1: "Ali", 2: "Sara"}  
print(users.get(1))
```

4. Code Patterns

Counting

```
words = ["a", "b", "a"]

count = {}
for w in words:
    count[w] = count.get(w, 0) + 1
```

Flatten

```
flat = [x for row in matrix for x in row]
```

5. Common Mistakes

- Using list instead of set for lookup
- Mutable default arguments
- Confusing copy with reference
- Nested mutable objects inside tuple
- Overusing complex comprehensions

6. Interview Questions with Answers

Q: Why are dict and set fast?

- ☐ Because of hashing ($O(1)$ operations)

Q: What is Python memory model?

- Objects + references

Q: Difference between == and is?

- == → value
- is → identity

Q: Why tuple as key?

- Immutable + hashable

7. Summary (Quick Revision)

- Python uses **objects and references**
- List → dynamic, mutable
- Tuple → immutable, hashable
- Set → unique, fast lookup
- Dict → key-value, core backend structure
- Mutable → same object changes
- Immutable → new object created
- Comprehensions → efficient data transformation

- Memory model:
 - Variables → references
 - Objects → stored in memory

Chapter 2: Functions & Scope

****(*args, kwargs, Closures, LEGB Rule, First-Class Functions)**

1. Introduction (Why This Matters in Backend)

Functions are the **core building blocks** of any backend system.

In real backend applications:

- Every API endpoint is a function
- Business logic is written inside functions
- Middleware, authentication, validation → all use functions
- Frameworks like FastAPI heavily rely on **function behavior**

□ If you don't understand functions deeply:

- You will struggle with API design
- You will write repetitive and messy code
- You will fail interview questions on Python internals

□ If you master this:

- You can write **clean, reusable, scalable backend code**
- You can understand advanced concepts like **decorators, dependency injection**
- You will confidently answer **interview questions**

2. Core Concepts

2.1 Functions in Python (Quick Foundation)

A function is a **block of reusable code**.

```
def greet(name):  
    return f"Hello {name}"
```

Key Points:

- Functions improve **code reusability**
- Functions reduce duplication
- Functions improve readability

2.2 *args (Variable Positional Arguments)

*args allows a function to accept **multiple positional arguments**.

Example:

```
def add_numbers(*args):  
    return sum(args)  
  
print(add_numbers(1, 2, 3)) # 6
```

Internal Understanding:

- args is a **tuple**
- You can pass any number of values

```
def check(*args):  
    print(type(args))  
  
check(1, 2, 3)  # <class 'tuple'>
```

When to Use:

- Unknown number of inputs
- Flexible APIs
- Logging functions

Backend Use Case:

```
def log_events(*events):  
    for event in events:  
        print(f"Logging: {event}")
```

2.3 **kwargs (Variable Keyword Arguments)

****kwargs** allows passing **named arguments**.

Example:

```
def create_user(**kwargs):  
    print(kwargs)  
  
create_user(name="Ali", age=25)
```

Internal Understanding:

- `kwargs` is a **dictionary**

```
def check(**kwargs):  
    print(type(kwargs))
```

```
check(a=1, b=2)  # <class 'dict'>
```

When to Use:

- Dynamic data input
- Configuration
- API payloads

Backend Use Case:

```
def update_user(**data):  
    for key, value in data.items():  
        print(f"{key} = {value}")
```

2.4 Combining `*args` and `**kwargs`

```
def func(a, *args, **kwargs):  
    print(a)  
    print(args)  
    print(kwargs)
```

```
func(1, 2, 3, x=10, y=20)
```

Order Rule (Important for Interview):

Normal arguments → *args → **kwargs

2.5 First-Class Functions (Very Important)

In Python, functions are **first-class citizens**.

□ This means:

- Functions can be assigned to variables
- Functions can be passed as arguments
- Functions can return other functions

Example:

```
def greet():  
    return "Hello"
```

```
func = greet  
print(func()) # Hello
```

Passing Function as Argument:

```
def execute(func):  
    return func()
```

```
def say_hi():  
    return "Hi"  
  
print(execute(say_hi))
```

Returning Function:

```
def outer():  
    def inner():  
        return "Inner function"  
    return inner  
  
f = outer()  
print(f())
```

Backend Importance:

- Used in decorators
- Used in middleware
- Used in dependency injection

2.6 Closures (Important Concept)

A closure is a function that **remembers variables from its outer scope**.

Example:

```
def outer(x):  
    def inner():  
        return x  
    return inner  
  
f = outer(10)  
print(f()) # 10
```

Key Idea:

- Inner function remembers x
- Even after outer function finishes

Real Understanding:

```
def counter():  
    count = 0  
  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
  
    return increment  
  
c = counter()  
print(c()) # 1  
print(c()) # 2
```

Backend Use Case:

- Maintaining state
- Rate limiting
- Caching

2.7 LEGB Rule (Scope Resolution)

Python resolves variables using the **LEGB rule**:

L → Local

E → Enclosing

G → Global

B → Built-in

Example:

```
x = "global"
```

```
def outer():  
    x = "enclosing"
```

```
    def inner():  
        x = "local"  
        print(x)
```

```
    inner()
```

```
outer()
```

Output:

local

Step-by-step Resolution:

1. Check Local scope
2. Then Enclosing
3. Then Global
4. Then Built-in

Example 2:

```
x = 10

def func():
    print(x)

func()  # 10
```

Using global keyword:

```
x = 10

def change():
    global x
    x = 20

change()
print(x)  # 20
```

Using nonlocal keyword:

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal x  
        x = 20  
  
    inner()  
    print(x)  
  
outer() # 20
```

Backend Importance:

- Avoid variable conflicts
- Control data flow
- Debug issues easily

3. Real-World Examples

Example 1: API Handler

```
def get_user(user_id):  
    return {"id": user_id}
```

Example 2: Dynamic Filters using `**kwargs`

```
def filter_users(**filters):  
    print(filters)  
  
filter_users(age=25, city="Delhi")
```

Example 3: Middleware Pattern

```
def middleware(func):  
    def wrapper():  
        print("Before")  
        result = func()  
        print("After")  
        return result  
    return wrapper
```

Example 4: Closure for State

```
def request_counter():  
    count = 0  
  
    def handle():  
        nonlocal count  
        count += 1  
        return count  
  
    return handle
```

4. Code Examples (Important Patterns)

Pattern 1: Flexible Function

```
def process_data(*args, **kwargs):  
    print(args)  
    print(kwargs)
```

Pattern 2: Function Dispatcher

```
def add(x, y):  
    return x + y  
  
def multiply(x, y):  
    return x * y  
  
operations = {  
    "add": add,  
    "mul": multiply  
}  
  
print(operations["add"](2, 3))
```

Pattern 3: Callback Function

```
def process(func):  
    return func(10)  
  
def square(x):
```

```
        return x * x

print(process(square))
```

Pattern 4: Closure Cache

```
def cache():
    data = {}

    def get(key):
        return data.get(key)

    def set(key, value):
        data[key] = value

    return get, set
```

5. Common Mistakes

✗ Confusing `*args` with list

□ It is a tuple

✗ Confusing `**kwargs` with list

□ It is a dictionary

✗ Wrong order of arguments

```
def func(*args, a): # ✗wrong
    pass
```

✗Forgetting `nonlocal` in closures

✗Overusing global variables

✗Not understanding scope → leads to bugs

6. Interview Questions with Answers

Q1: What is `*args`?

Answer:

Allows multiple positional arguments, stored as tuple

Q2: What is `**kwargs`?

Answer:

Allows multiple keyword arguments, stored as dictionary

Q3: Difference between `args` and `kwargs`?

- `args` → positional

- kwargs → named

Q4: What are first-class functions?

Functions treated as objects (can pass, return, assign)

Q5: What is closure?

Function remembering outer variable even after outer function ends

Q6: What is LEGB rule?

Scope resolution order:

Local → Enclosing → Global → Built-in

Q7: What does this print?

```
x = 10

def func():
    x = 20

func()
print(x)
```

Answer: 10 (global unchanged)

Q8: Why use closures?

- Maintain state
- Avoid global variables

7. Summary (Quick Revision)

- Functions = reusable logic blocks
- `*args` → multiple positional → tuple
- `**kwargs` → multiple named → dictionary
- First-class functions → can pass, return, assign
- Closures → remember outer variables
- LEGB → Local → Enclosing → Global → Built-in
- Use:
 - closures for state
 - kwargs for flexibility
 - functions for modular design

Chapter 3: Decorators (Interview Favorite)

(Basic Decorators, Decorators with Arguments, Real-World Use Cases)

1. Introduction (Why This Matters in Backend)

Decorators are one of the **most important and most asked topics** in Python backend interviews.

In real backend systems, decorators are used everywhere:

- Authentication (`@login_required`)
- Authorization (`@admin_only`)
- Logging (`@log_request`)
- Rate limiting
- Caching
- Validation

Frameworks like **FastAPI** and **Flask** internally use decorators for:

- Routing (`@app.get()`)
- Middleware
- Dependency injection

☐ If you don't understand decorators:

- You won't understand how frameworks work
- You will struggle in interviews
- You will write repetitive code

☐ If you master decorators:

- You write **clean and reusable backend code**
- You can build your own frameworks

- You will answer **interview questions easily**

2. Core Concepts

2.1 What is a Decorator?

A decorator is a **function that modifies another function** without changing its code.

□ Simple idea:

- Take a function
- Add extra behavior
- Return a new function

Basic Example (Without Decorator)

```
def greet():  
    print("Hello")  
  
def wrapper():  
    print("Before")  
    greet()  
    print("After")  
  
wrapper()
```

Problem:

- Code repetition
- Not reusable

2.2 Basic Decorator (Core Concept)

Step-by-step:

```
def decorator(func):  
    def wrapper():  
        print("Before function")  
        func()  
        print("After function")  
    return wrapper
```

Applying Decorator:

```
def greet():  
    print("Hello")  
  
greet = decorator(greet)  
  
greet()
```

Cleaner Syntax (Important)

```
@decorator
def greet():
    print("Hello")
```

Output:

```
Before function
Hello
After function
```

Key Idea:

- `@decorator = greet = decorator(greet)`

2.3 Decorator with Arguments (Important)

Problem:

Basic decorator only works with functions having no arguments.

Solution:

```
def decorator(func):
    def wrapper(*args, **kwargs):
        print("Before")
        result = func(*args, **kwargs)
        print("After")
```

```
        return result
    return wrapper
```

Example:

```
@decorator
def add(a, b):
    return a + b

print(add(2, 3))
```

Why Important?

- Works with any function
- Required in real backend systems

2.4 Decorator with Parameters (Advanced)

Sometimes decorators themselves need arguments.

Example:

```
def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                func(*args, **kwargs)
        return wrapper
```

return decorator

Usage:

```
@repeat(3)
def say_hi():
    print("Hi")

say_hi()
```

Key Idea:

□ 3 layers:

1. Outer function → takes decorator arguments
2. Decorator function → takes function
3. Wrapper → executes function

2.5 Preserving Function Metadata (Important)

Problem:

Decorators overwrite function name and docstring.

Example:

```
def decorator(func):
    def wrapper():
        return func()
```

```
        return wrapper

@decorator
def greet():
    """This is greet"""
    return "Hello"

print(greet.__name__) # wrapper ✕
```

Solution:

```
from functools import wraps

def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

- Always use `@wraps` in production code

2.6 Real Understanding (Behind the Scene)

Decorator works like:

```
greet = decorator(greet)
```

So:

- Original function is replaced
- Wrapper controls execution

3. Real-World Examples (Backend)

Example 1: Logging Decorator

```
def log(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Finished {func.__name__}")
        return result
    return wrapper

@log
def process():
    print("Processing data")

process()
```

Example 2: Authentication Decorator

```
def require_auth(func):
    def wrapper(user, *args, **kwargs):
        if not user.get("is_authenticated"):
            return "Unauthorized"
        return func(user, *args, **kwargs)
    return wrapper
```

```
@require_auth
def get_profile(user):
    return f"Welcome {user['name']}"
```

Example 3: Timing Decorator

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print("Time:", end - start)
        return result
    return wrapper
```

Example 4: Caching Decorator

```
def cache(func):
    memory = {}

    def wrapper(x):
        if x in memory:
            return memory[x]
        result = func(x)
        memory[x] = result
        return result
```

```
return wrapper
```

4. Code Examples (Important Patterns)

Pattern 1: Input Validation

```
def validate(func):  
    def wrapper(x):  
        if x < 0:  
            return "Invalid input"  
        return func(x)  
    return wrapper
```

Pattern 2: Role-Based Access

```
def admin_only(func):  
    def wrapper(user):  
        if user.get("role") != "admin":  
            return "Access denied"  
        return func(user)  
    return wrapper
```

Pattern 3: Retry Logic

```
def retry(func):  
    def wrapper(*args, **kwargs):  
        for _ in range(3):
```

```
        try:
            return func(*args, **kwargs)
        except:
            continue
    return wrapper
```

Pattern 4: Rate Limiting (Basic Idea)

```
def rate_limit(func):
    calls = 0

    def wrapper(*args, **kwargs):
        nonlocal calls
        if calls > 5:
            return "Too many requests"
        calls += 1
        return func(*args, **kwargs)

    return wrapper
```

5. Common Mistakes

✗ Forgetting `*args, **kwargs`

→ breaks function

✗ Not returning result

✗ Not using `@wraps`

✗ Confusing decorator levels

✗ Overusing decorators → complex code

6. Interview Questions with Answers

Q1: What is a decorator?

Function that modifies another function without changing its code

Q2: How decorators work internally?

```
func = decorator(func)
```

Q3: Why use decorators?

- Code reuse
- Separation of concerns
- Cleaner code

Q4: What is wrapper function?

Function inside decorator that adds extra behavior

Q5: Why use *args and **kwargs?

To support any function signature

Q6: What is @wraps?

Preserves metadata of original function

Q7: Can decorators take arguments?

Yes, using nested functions

Q8: Real-world use?

- Authentication
- Logging
- Caching
- Rate limiting

7. Summary (Quick Revision)

- Decorator = function modifying another function

- Syntax:

```
@decorator  
def func():
```

- Use `*args`, `**kwargs` for flexibility
- Use `@wraps` for metadata
- Types:
 - Basic decorator
 - Decorator with arguments
 - Decorator with parameters
- Backend uses:
 - Auth
 - Logging
 - Caching
 - Middleware

Chapter 4: Object-Oriented Programming (OOP) in Python

(Classes, Inheritance, MRO, Encapsulation, Polymorphism, Dataclasses)

1. Introduction (Why This Matters in Backend)

In backend development, applications grow quickly:

- Many modules
- Complex business logic
- Multiple APIs
- Database layers

To manage this complexity, we use **Object-Oriented Programming (OOP)**.

□ OOP helps in:

- Structuring large applications
- Writing reusable code
- Managing data and behavior together
- Designing scalable systems

In real backend systems:

- Models → classes
- Services → classes
- Controllers → classes
- ORM (SQLAlchemy) → class-based

□ If you don't understand OOP:

- Your code becomes messy and hard to scale
- You struggle with frameworks

- You fail system design + backend interviews

□ If you master OOP:

- You can design **clean architectures**
- You understand frameworks deeply
- You can explain projects confidently in interviews

2. Core Concepts

2.1 Classes and Objects (Foundation)

□ Class

A class is a **blueprint** for creating objects.

```
class User:  
    pass
```

□ Object

An object is an **instance of a class**.

```
user1 = User()
```

□ Class with Attributes

```
class User:  
    def __init__(self, name, age):
```

```
        self.name = name
        self.age = age

user1 = User("Ali", 25)
print(user1.name)
```

Key Points:

- `__init__` → constructor
- `self` → refers to current object

Backend Use Case:

- Represent users, orders, products
- Encapsulate data + behavior

2.2 Methods (Behavior in Classes)

Instance Method

```
class User:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hello {self.name}"
```

Class Method

```
class User:
    count = 0

    @classmethod
    def increment(cls):
        cls.count += 1
```

Static Method

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
```

Differences:

Type	Uses	Access
Instance	Object data	self
Class method	Class data	cls
Static	Utility	none

2.3 Encapsulation (Data Protection)

Encapsulation means **hiding internal data** and controlling access.

Example:

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance # protected

    def deposit(self, amount):
        self._balance += amount

    def get_balance(self):
        return self._balance
```

Types of Access:

Type	Syntax	Meaning
Public	x	Accessible
Protected	_x	Internal use
Private	__x	Strong hiding

Why Important:

- Prevent accidental changes
- Secure data
- Maintain integrity

2.4 Inheritance (Code Reuse)

Inheritance allows one class to **reuse another class's properties**.

Example:

```
class Animal:
    def speak(self):
        return "Sound"

class Dog(Animal):
    def speak(self):
        return "Bark"
```

Types of Inheritance:

- Single
- Multiple
- Multilevel

Backend Use Case:

- Base models
- Shared logic
- API base classes

2.5 Method Resolution Order (MRO)

MRO defines the order in which Python searches methods.

Example:

```
class A:
    def show(self):
        print("A")

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass

print(D.mro())
```

Key Idea:

- Python uses **C3 Linearization**
- Searches from left to right

Why Important:

- Avoid confusion in multiple inheritance
- Debug method conflicts

2.6 Polymorphism (Same Interface, Different Behavior)

Polymorphism means:

- Same method name → different behavior

Example:

```
class Dog:
    def speak(self):
        return "Bark"

class Cat:
    def speak(self):
        return "Meow"

def animal_sound(animal):
    print(animal.speak())
```

Output:

```
animal_sound(Dog())
animal_sound(Cat())
```

Backend Use Case:

- Different services with same interface
- Plug-and-play architecture

2.7 Dataclasses (Modern Python Feature)

Dataclasses simplify class creation.

Without Dataclass:

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

With Dataclass:

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int
```

Benefits:

- Less boilerplate
- Auto-generated methods (`__init__`, `__repr__`)
- Cleaner code

Backend Use Case:

- Data transfer objects (DTOs)
- API schemas
- Config objects

3. Real-World Examples

Example 1: User Model

```
class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    def display(self):
        return f"{self.name} ({self.email})"
```

Example 2: Service Layer

```
class UserService:
    def get_user(self, user_id):
        return {"id": user_id}
```

Example 3: Base API Class

```
class BaseAPI:
    def log(self):
        print("Logging request")

class UserAPI(BaseAPI):
    def get(self):
        return "User data"
```

Example 4: Polymorphic Processing

```
class Payment:
    def process(self):
        pass

class CardPayment(Payment):
    def process(self):
        return "Card processed"

class UPI(Payment):
    def process(self):
        return "UPI processed"
```

4. Code Examples (Important Patterns)

Pattern 1: Factory Pattern

```
class User:
    def __init__(self, name):
        self.name = name

def create_user(name):
    return User(name)
```

Pattern 2: Singleton (Basic Idea)

```
class Singleton:
    _instance = None

    def __new__(cls):
        if not cls._instance:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Pattern 3: Composition

```
class Engine:
    def start(self):
        return "Engine started"

class Car:
    def __init__(self):
        self.engine = Engine()
```

Pattern 4: Method Override

```
class Base:
    def show(self):
        return "Base"

class Child(Base):
    def show(self):
        return "Child"
```

5. Common Mistakes

✗ Forgetting `self`

✗ Overusing inheritance

✗ Not understanding MRO

✗ Using class variables incorrectly

✗ Not using encapsulation

✗ Writing everything in one class

6. Interview Questions with Answers

Q1: What is OOP?

Programming using classes and objects

Q2: Difference between class and object?

- Class → blueprint

- Object → instance

Q3: What is inheritance?

Reusing properties of parent class

Q4: What is polymorphism?

Same method, different behavior

Q5: What is encapsulation?

Hiding data and controlling access

Q6: What is MRO?

Order in which methods are resolved

Q7: What is dataclass?

Simplified class with auto-generated methods

Q8: Why use OOP in backend?

- Scalability
- Maintainability
- Clean structure

7. Summary (Quick Revision)

- Class → blueprint
- Object → instance
- Encapsulation → protect data
- Inheritance → reuse code
- Polymorphism → flexible behavior
- MRO → method lookup order
- Dataclass → cleaner class creation

Chapter 5: Async Programming in Python

(Event Loop, async/await, asyncio vs Threading vs Multiprocessing)

1. Introduction (Why This Matters in Backend)

Modern backend systems handle:

- Thousands of users

- Multiple API requests at the same time
- Database calls
- External API calls

If your backend processes requests **one by one**, it becomes:

- Slow
- Unresponsive
- Not scalable

□ This is where **asynchronous programming (async)** comes in.

Frameworks like **FastAPI** are fast because they use async.

Real Backend Scenario:

Imagine:

- 100 users request data
- Each request waits for database (1 second)

Without async:

□ Total time $\approx$ 100 seconds

With async:

□ Total time $\approx$ ~1 second (concurrent execution)

□ If you don't understand async:

- You cannot build high-performance APIs
- You will fail backend interviews

□ If you master async:

- You build **fast and scalable systems**
- You understand modern backend frameworks
- You stand out in interviews

2. Core Concepts

2.1 Synchronous vs Asynchronous Programming

□ Synchronous (Blocking)

Tasks run **one after another**

```
import time
```

```
def task():  
    time.sleep(2)  
    print("Done")
```

```
task()
```

```
task()
```

□ Total time = 4 seconds

□ Asynchronous (Non-Blocking)

Tasks run **concurrently**

```
import asyncio
```

```
async def task():
    await asyncio.sleep(2)
    print("Done")
```

Key Idea:

- `sleep()` → blocks execution
- `await` → allows other tasks to run

2.2 Event Loop (Heart of Async)

The event loop is the **engine that runs async code**.

□ It:

- Schedules tasks
- Switches between tasks
- Executes coroutines

Simple Example:

```
import asyncio

async def main():
    print("Start")
    await asyncio.sleep(1)
    print("End")

asyncio.run(main())
```

Internal Concept:

- Tasks are queued
- Event loop runs them efficiently

Backend Insight:

- FastAPI runs on event loop
- Handles multiple requests concurrently

2.3 async and await (Core Syntax)

async

Defines a coroutine (async function)

```
async def fetch():  
    return "data"
```

await

Used to pause execution until result is ready

```
async def main():  
    result = await fetch()
```

Key Rules:

- `await` only inside `async def`
- Always `await` async calls

2.4 Running Multiple Tasks (Concurrency)

Example:

```
import asyncio

async def task1():
    await asyncio.sleep(2)
    print("Task 1 done")

async def task2():
    await asyncio.sleep(2)
    print("Task 2 done")

async def main():
    await asyncio.gather(task1(), task2())

asyncio.run(main())
```

Output:

- Both tasks finish in ~2 seconds

Key Idea:

- Concurrency $\neq$ Parallelism
- Tasks run “together” but not truly simultaneously

2.5 asyncio (Async Framework in Python)

asyncio is Python’s built-in async library.

Features:

- Event loop
- Coroutines
- Task scheduling

Example:

```
import asyncio

async def main():
    print("Start")
    await asyncio.sleep(1)
    print("End")

asyncio.run(main())
```

Backend Use Cases:

- API calls

- Database queries
- File I/O
- Network operations

2.6 Threading vs Multiprocessing vs Async

□ Threading

- Multiple threads
- Shared memory
- Good for I/O tasks

```
import threading
```

□ Multiprocessing

- Multiple processes
- Separate memory
- Good for CPU-heavy tasks

```
import multiprocessing
```

□ Async

- Single thread
- Event loop
- Best for I/O tasks

Comparison Table:

Feature	Threading	Multiprocessing	Async
Memory	Shared	Separate	Shared
Best for	I/O	CPU	I/O
Complexity	Medium	High	Low
Performance	Medium	High	High

Backend Insight:

- Async → APIs, DB calls
- Threading → legacy systems
- Multiprocessing → heavy computation

2.7 Blocking vs Non-Blocking (Critical Concept)

Blocking Code:

```
import time

time.sleep(2)  # blocks
```

Non-Blocking Code:

```
await asyncio.sleep(2)  # non-blocking
```

Backend Importance:

- Blocking → slows entire server
- Non-blocking → handles multiple users

3. Real-World Examples

Example 1: API Call Simulation

```
import asyncio

async def fetch_data():
    await asyncio.sleep(2)
    return "data"
```

Example 2: Multiple Requests

```
async def main():
    results = await asyncio.gather(
        fetch_data(),
        fetch_data(),
        fetch_data()
    )
```

Example 3: FastAPI Style

```
async def get_user():  
    return {"name": "Ali"}
```

Example 4: Database Simulation

```
async def fetch_from_db():  
    await asyncio.sleep(1)  
    return "DB data"
```

4. Code Examples (Important Patterns)

Pattern 1: Parallel API Calls

```
async def main():  
    tasks = [fetch_data() for _ in range(5)]  
    results = await asyncio.gather(*tasks)
```

Pattern 2: Timeout Handling

```
try:  
    await asyncio.wait_for(fetch_data(), timeout=2)  
except asyncio.TimeoutError:  
    print("Timeout")
```

Pattern 3: Background Task

```
async def background():  
    while True:  
        print("Running")  
        await asyncio.sleep(1)
```

Pattern 4: Async Queue

```
queue = asyncio.Queue()  
await queue.put("task")
```

5. Common Mistakes

✗ Using `time.sleep()` in async code

- blocks event loop

✗ Forgetting `await`

✗ Mixing sync and async incorrectly

✗ CPU-heavy tasks in async

- should use multiprocessing

✗ Not understanding concurrency vs parallelism

6. Interview Questions with Answers

Q1: What is async programming?

Non-blocking execution of tasks

Q2: What is event loop?

Engine that runs async tasks

Q3: Difference between async and threading?

- Async → single thread
- Threading → multiple threads

Q4: What is await?

Pauses execution until result is ready

Q5: What is asyncio?

Python library for async programming

Q6: When to use async?

- I/O tasks
- API calls
- DB operations

Q7: What is blocking vs non-blocking?

- Blocking → stops execution
- Non-blocking → allows other tasks

Q8: Can async run CPU-heavy tasks?

No, use multiprocessing

7. Summary (Quick Revision)

- Async = non-blocking execution
- Event loop = manages tasks
- `async` → defines coroutine
- `await` → waits without blocking

- Use `asyncio.gather()` for concurrency
- Async best for:
 - API calls
 - DB queries
 - Network tasks
- Avoid:
 - blocking calls
 - missing `await`

Chapter 6: Error Handling & Logging

(try/except, Custom Exceptions, Logging System)

1. Introduction (Why This Matters in Backend)

Backend systems run continuously and handle:

- User requests
- Database operations
- External API calls
- File processing

In real-world systems, **errors are unavoidable**:

- Database connection fails
- Invalid user input
- API timeout
- Unexpected bugs

□ If errors are not handled properly:

- Your application crashes
- Users get poor experience
- Debugging becomes difficult

□ Logging is equally important:

- Helps track issues
- Helps monitor system behavior
- Helps in production debugging

Real Backend Scenario:

A user sends wrong data:

- Without error handling → server crashes
- With error handling → meaningful response returned

□ Backend goal:

- ✓ Handle errors gracefully
- ✓ Never crash the system
- ✓ Log everything important

2. Core Concepts

2.1 What is an Exception?

An exception is an **error that occurs during program execution**.

Example:

```
x = 10 / 0 # ZeroDivisionError
```

Common Exceptions:

Exception	Cause
ZeroDivisionError	Divide by zero
ValueError	Invalid value
TypeError	Wrong data type

KeyError	Missing dictionary key
IndexError	Invalid list index

2.2 try / except (Basic Error Handling)

Syntax:

```
try:
    risky_code
except Exception:
    handle_error
```

Example:

```
try:
    x = int("abc")
except ValueError:
    print("Invalid number")
```

Multiple Exceptions:

```
try:
    x = int("abc")
except ValueError:
    print("Value error")
except TypeError:
    print("Type error")
```

Catch All Exception:

```
try:
    x = 10 / 0
except Exception as e:
    print(e)
```

Backend Insight:

- Always catch specific exceptions first
- Use general exception only when needed

2.3 else and finally (Advanced Flow Control)

else block

Runs when no exception occurs

```
try:
    x = 10 / 2
except:
    print("Error")
else:
    print("Success")
```

finally block

Runs always (cleanup code)

```
try:
    file = open("test.txt")
except:
    print("Error")
finally:
    print("Closing resources")
```

Backend Use:

- Closing DB connections
- Releasing resources
- Logging completion

2.4 Raising Exceptions (raise keyword)

You can manually raise exceptions.

Example:

```
def withdraw(amount):
    if amount < 0:
        raise ValueError("Invalid amount")
```

Why Important:

- Enforce business rules
- Validate data

2.5 Custom Exceptions (Very Important)

Custom exceptions allow creating **meaningful error types**.

Example:

```
class InsufficientBalance(Exception):  
    pass  
  
def withdraw(balance, amount):  
    if amount > balance:  
        raise InsufficientBalance("Not enough money")
```

Backend Use Cases:

- Payment errors
- Authentication errors
- Validation errors

2.6 Logging (Critical in Production)

Printing is not enough in backend systems.

- ☐ Use logging instead

Basic Logging:

```
import logging

logging.basicConfig(level=logging.INFO)

logging.info("App started")
logging.error("Something failed")
```

Logging Levels:

Level	Purpose
DEBUG	Detailed info
INFO	General info
WARNING	Potential issue
ERROR	Error occurred
CRITICAL	Serious failure

Example:

```
logging.debug("Debug info")
logging.warning("Warning message")
```

2.7 Logging to File

```
logging.basicConfig(
    filename="app.log",
    level=logging.INFO
)
```

```
logging.info("Saved to file")
```

2.8 Structured Logging (Backend Best Practice)

Instead of plain text:

```
logging.info(f"User {user_id} logged in")
```

Better approach:

- Include context
- Use JSON logging (advanced systems)

2.9 Exception Handling + Logging Together

Example:

```
import logging

def divide(a, b):
    try:
        return a / b
    except Exception as e:
        logging.error(f"Error: {e}")
        return None
```

Backend Insight:

- Always log errors
- Never silently ignore exceptions

3. Real-World Examples

Example 1: API Error Handling

```
def get_user(user_id):  
    try:  
        return {"id": user_id}  
    except Exception:  
        return {"error": "User not found"}
```

Example 2: Database Error Handling

```
def fetch_data():  
    try:  
        # simulate DB call  
        raise Exception("DB error")  
    except Exception as e:  
        logging.error(e)
```

Example 3: Validation

```
def validate_age(age):  
    if age < 0:
```

```
raise ValueError("Age cannot be negative")
```

Example 4: Logging User Activity

```
def login(user):  
    logging.info(f"{user} logged in")
```

4. Code Examples (Important Patterns)

Pattern 1: Safe Execution Wrapper

```
def safe_execute(func):  
    try:  
        return func()  
    except Exception as e:  
        logging.error(e)
```

Pattern 2: Retry Mechanism

```
def retry(func):  
    for _ in range(3):  
        try:  
            return func()  
        except:  
            continue
```

Pattern 3: Input Validation

```
def process(age):  
    if not isinstance(age, int):  
        raise TypeError("Age must be integer")
```

Pattern 4: Graceful API Response

```
def api_handler():  
    try:  
        return {"data": "success"}  
    except:  
        return {"error": "failed"}
```

5. Common Mistakes

✗ Using bare except

```
except:  
    pass # ✗ hides errors
```

✗ Not logging errors

✗ Catching too broad exceptions

✗Ignoring exceptions silently

✗Using print instead of logging

6. Interview Questions with Answers

Q1: What is exception handling?

Handling runtime errors without crashing program

Q2: Difference between error and exception?

- Error → severe problem
- Exception → can be handled

Q3: What is finally block?

Always executes, used for cleanup

Q4: What is raise?

Used to manually throw exception

Q5: Why use custom exceptions?

Better readability and control

Q6: Why logging is important?

- Debugging
- Monitoring
- Production tracking

Q7: Difference between logging and print?

- logging → structured, configurable
- print → simple output

Q8: What are logging levels?

DEBUG, INFO, WARNING, ERROR, CRITICAL

7. Summary (Quick Revision)

- Exception = runtime error
- Use:
 - try → risky code
 - except → handle error
 - finally → cleanup
- Raise exceptions for validation

- Custom exceptions improve clarity
- Logging is essential for backend
- Use logging instead of print
- Never ignore errors silently

Chapter 7: Type Hints & Pydantic

(Type Annotations, Data Validation, Pydantic Models)

1. Introduction (Why This Matters in Backend)

Backend systems deal with **data everywhere**:

- API requests
- Database records
- JSON payloads
- User inputs

If data is not validated properly:

- Bugs increase
- Security risks appear
- APIs break

□ Type hints and validation help:

- Write **clean and predictable code**
- Catch errors early
- Improve readability
- Enable better tooling (IDE, linters)

□ Pydantic (used in FastAPI) is one of the most powerful tools for:

- Data validation
- Data parsing
- Schema definition

Real Backend Scenario:

User sends API request:

```
{  
  "name": "Ali",  
  "age": "twenty"  
}
```

Without validation:

✗ System may crash

With Pydantic:

✓ Error handled automatically

☐ If you don't understand this:

- You cannot build production APIs
- You will struggle with FastAPI

☐ If you master this:

- You write **safe and reliable backend systems**
- You pass backend interviews easily

2. Core Concepts

2.1 Type Hints (Type Annotations)

Type hints allow you to **specify data types** of variables and functions.

Basic Example:

```
def add(a: int, b: int) -> int:  
    return a + b
```

Key Points:

- `a: int` → parameter type
- `-> int` → return type

Important:

- ☐ Python does NOT enforce types at runtime
- ☐ It is used for:
 - Readability
 - Static checking
 - IDE support

Example:

```
def greet(name: str) -> str:  
    return f"Hello {name}"
```

Backend Use:

- API inputs

- Function contracts
- Data processing

2.2 Common Type Hints

Basic Types:

```
x: int = 10
name: str = "Ali"
price: float = 10.5
```

List:

```
from typing import List

numbers: List[int] = [1, 2, 3]
```

Dictionary:

```
from typing import Dict

user: Dict[str, int] = {"age": 25}
```

Tuple:

```
from typing import Tuple

coords: Tuple[int, int] = (10, 20)
```

Optional:

```
from typing import Optional

name: Optional[str] = None
```

Union:

```
from typing import Union

value: Union[int, str]
```

Backend Insight:

- Helps define API schemas
- Improves maintainability

2.3 Type Hinting Functions (Advanced)

Default Values:

```
def greet(name: str = "Guest") -> str:  
    return f"Hello {name}"
```

Multiple Types:

```
def process(value: int | str):  
    print(value)
```

Function as Type:

```
from typing import Callable  
  
def execute(func: Callable[[int], int]):  
    return func(10)
```

2.4 Why Type Hints Matter in Backend

Benefits:

- Better readability
- Easy debugging
- Early error detection
- IDE auto-completion
- Clean API design

Example (Without Type Hint):

```
def process(data):  
    return data + 1
```

With Type Hint:

```
def process(data: int) -> int:  
    return data + 1
```

- Much clearer for developers

2.5 Pydantic (Core Backend Tool)

Pydantic is used for:

- Data validation
- Data parsing
- Type enforcement

Basic Example:

```
from pydantic import BaseModel  
  
class User(BaseModel):  
    name: str  
    age: int
```

Usage:

```
user = User(name="Ali", age=25)
print(user.name)
```

Invalid Data:

```
User(name="Ali", age="abc") # ✗error
```

- ❑ Pydantic automatically validates

2.6 Default Values and Validation

```
class User(BaseModel):
    name: str
    age: int = 18
```

Example:

```
user = User(name="Ali")
print(user.age) # 18
```

2.7 Field Validation (Advanced)

```
from pydantic import BaseModel, Field

class User(BaseModel):
    age: int = Field(gt=0)
```

Meaning:

- `gt=0` → age must be greater than 0

2.8 Custom Validation

```
from pydantic import validator

class User(BaseModel):
    name: str

    @validator("name")
    def validate_name(cls, v):
        if len(v) < 3:
            raise ValueError("Too short")
        return v
```

2.9 Nested Models

```
class Address(BaseModel):
    city: str
```

```
class User(BaseModel):  
    name: str  
    address: Address
```

2.10 Pydantic in FastAPI (Real Use)

```
from fastapi import FastAPI  
  
app = FastAPI()  
  
class User(BaseModel):  
    name: str  
    age: int  
  
@app.post("/user")  
def create_user(user: User):  
    return user
```

□ FastAPI automatically:

- Validates input
- Returns error if invalid

3. Real-World Examples

Example 1: API Request Validation

```
class Login(BaseModel):  
    username: str  
    password: str
```

Example 2: E-commerce Product

```
class Product(BaseModel):  
    name: str  
    price: float
```

Example 3: Optional Fields

```
class User(BaseModel):  
    name: str  
    email: Optional[str] = None
```

Example 4: Response Model

```
class Response(BaseModel):  
    status: str  
    data: dict
```

4. Code Examples (Important Patterns)

Pattern 1: Safe Input Handling

```
def process(age: int):  
    return age + 1
```

Pattern 2: Data Transformation

```
class User(BaseModel):  
    name: str  
  
user = User(name="Ali")  
print(user.dict())
```

Pattern 3: Validation Layer

```
def validate(user: User):  
    return True
```

Pattern 4: Default Config

```
class Config(BaseModel):  
    debug: bool = False
```

5. Common Mistakes

✗ Thinking type hints enforce types

✗ Not using Pydantic in APIs

✗ Overcomplicating validation

✗ Forgetting Optional

✗ Not validating nested data

6. Interview Questions with Answers

Q1: What are type hints?

Annotations for variable types

Q2: Are type hints enforced?

No (unless using tools)

Q3: What is Pydantic?

Library for data validation and parsing

Q4: Why Pydantic is used in FastAPI?

Automatic validation of request data

Q5: What is Optional?

Value can be None

Q6: Difference between Union and Optional?

- Optional = Union[type, None]

Q7: What is BaseModel?

Base class for Pydantic models

Q8: What happens on invalid data?

Validation error is raised

7. Summary (Quick Revision)

- Type hints improve readability
- Not enforced at runtime

- Used for:
 - API design
 - Debugging
 - Tooling
- Pydantic = validation engine
- BaseModel → define schema
- Supports:
 - Validation
 - Defaults
 - Nested models

Chapter 8: Testing with Pytest

(Fixtures, Mocking, Testing Async Code)

1. Introduction (Why This Matters in Backend)

In backend development, writing code is **not enough**.

You must ensure your code:

- Works correctly
- Handles edge cases
- Does not break when updated

□ This is where **testing** comes in.

Real Backend Problems Without Testing:

- API breaks after small change
- Bugs reach production
- Difficult to debug issues
- No confidence in code

With Testing:

- ✓ Detect bugs early
- ✓ Ensure reliability
- ✓ Improve code quality
- ✓ Build confidence during deployment

- Companies expect backend developers to know testing.
- Pytest is the **most popular testing framework in Python**.

2. Core Concepts

2.1 What is Testing?

Testing means **verifying that your code works as expected**.

Simple Example:

```
def add(a, b):  
    return a + b  
  
def test_add():  
    assert add(2, 3) == 5
```

Key Idea:

- `assert` checks correctness
- If condition fails → test fails

2.2 Why Pytest?

Pytest is preferred because:

- Simple syntax
- Powerful features
- Easy to write tests
- Supports fixtures and mocking

Running Tests:

pytest

2.3 Writing Basic Tests

Example:

```
def multiply(a, b):  
    return a * b  
  
def test_multiply():  
    assert multiply(2, 3) == 6
```

Multiple Tests:

```
def test_case1():  
    assert 1 + 1 == 2  
  
def test_case2():  
    assert 2 * 2 == 4
```

2.4 Fixtures (Very Important)

Fixtures provide **setup data for tests**.

Example:

```
import pytest

@pytest.fixture
def sample_user():
    return {"name": "Ali", "age": 25}

def test_user(sample_user):
    assert sample_user["name"] == "Ali"
```

Key Idea:

- Reusable setup
- Cleaner code

Fixture Scope:

```
@pytest.fixture(scope="module")
```

Types:

- function (default)

- module
- session

Backend Use:

- Database setup
- API client
- Test data

2.5 Parametrized Tests

Run same test with multiple inputs.

Example:

```
import pytest

@pytest.mark.parametrize("a,b,result", [
    (1, 2, 3),
    (2, 3, 5),
    (5, 5, 10)
])
def test_add(a, b, result):
    assert a + b == result
```

Benefit:

- Covers multiple cases
- Reduces code duplication

2.6 Mocking (Very Important in Backend)

Mocking is used to **simulate external dependencies**.

Why Needed?

- Avoid real DB calls
- Avoid external APIs
- Faster tests

Example:

```
from unittest.mock import Mock

def test_mock():
    mock_func = Mock(return_value=10)
    assert mock_func() == 10
```

Mocking API:

```
def fetch_data(api):
    return api()

def test_fetch():
    mock_api = Mock(return_value="data")
    assert fetch_data(mock_api) == "data"
```

Backend Use:

- Mock database
- Mock HTTP calls
- Mock services

2.7 Testing Async Code

Async code needs special handling.

Example:

```
import pytest
import asyncio

async def fetch():
    return "data"

@pytest.mark.asyncio
async def test_fetch():
    result = await fetch()
    assert result == "data"
```

Key Point:

- Use `@pytest.mark.asyncio`
- Use `await`

2.8 Testing Exceptions

Example:

```
import pytest

def divide(a, b):
    return a / b

def test_divide_error():
    with pytest.raises(ZeroDivisionError):
        divide(10, 0)
```

Backend Use:

- Validate error handling
- Ensure proper exceptions

2.9 Test Structure (Best Practice)

Project Structure:

```
project/
|
├── app/
│   └── main.py
|
└── tests/
```

| └─ test_main.py

Naming Rules:

- Test file → `test_*.py`
- Test function → `test_*`

2.10 Code Coverage (Advanced)

Check how much code is tested.

Command:

```
pytest --cov=app
```

Goal:

- Cover critical logic
- Avoid untested code

3. Real-World Examples

Example 1: API Testing

```
def get_user():  
    return {"name": "Ali"}  
  
def test_get_user():  
    assert get_user()["name"] == "Ali"
```

Example 2: Database Mock

```
def get_data(db):  
    return db.query()  
  
def test_db():  
    mock_db = Mock()  
    mock_db.query.return_value = [1, 2]  
    assert get_data(mock_db) == [1, 2]
```

Example 3: Validation Testing

```
def validate(age):  
    if age < 0:  
        raise ValueError  
  
def test_validate():  
    with pytest.raises(ValueError):  
        validate(-1)
```

Example 4: Async API Test

```
@pytest.mark.asyncio
async def test_api():
    result = await fetch()
    assert result == "data"
```

4. Code Examples (Important Patterns)

Pattern 1: Setup Fixture

```
@pytest.fixture
def db():
    return {"data": []}
```

Pattern 2: Dependency Injection Testing

```
def service(repo):
    return repo.get()

def test_service():
    mock_repo = Mock()
    mock_repo.get.return_value = "data"
    assert service(mock_repo) == "data"
```

Pattern 3: Edge Case Testing

```
def test_zero():  
    assert add(0, 0) == 0
```

Pattern 4: Error Handling Test

```
def test_error():  
    with pytest.raises(Exception):  
        raise Exception("error")
```

5. Common Mistakes

✗ Not writing tests

✗ Testing only happy path

✗ Not mocking external services

✗ Ignoring edge cases

✗ Mixing test and production code

✗ Not testing async properly

6. Interview Questions with Answers

Q1: What is pytest?

Python testing framework

Q2: What is fixture?

Reusable setup for tests

Q3: What is mocking?

Simulating dependencies

Q4: How to test async code?

Use `pytest.mark.asyncio`

Q5: What is assert?

Checks expected result

Q6: Why testing is important?

- Reliability
- Bug detection
- Confidence

Q7: What is parametrization?

Run test with multiple inputs

Q8: What is code coverage?

Measure of tested code

7. Summary (Quick Revision)

- Testing ensures code correctness
- Pytest = simple + powerful
- Use:
 - assert
 - fixtures
 - mocking
- Test:

- normal cases
 - edge cases
 - error cases
- Async testing → use `@pytest.mark.asyncio`
- Mock external systems

Chapter 9: Top Python Interview Questions

(GIL, Generators vs Iterators, Memory Management)

1. Introduction (Why This Matters in Backend)

In backend interviews, companies don't just test coding—they test your **deep understanding of Python internals**.

Common topics:

- GIL (Global Interpreter Lock)
- Generators & Iterators
- Memory management

□ These topics are:

- Frequently asked in interviews
- Critical for writing efficient backend code
- Important for performance optimization

Real Backend Impact:

- Poor memory handling → crashes
- Wrong use of generators → slow APIs
- Misunderstanding GIL → wrong concurrency decisions

□ If you master this:

- You stand out in interviews
- You write optimized code

- You understand how Python works internally

2. Core Concepts

2.1 GIL (Global Interpreter Lock)

□ What is GIL?

GIL is a **lock that allows only one thread to execute Python bytecode at a time.**

Simple Explanation:

Even if you create multiple threads:

- Only one thread runs at a time in Python

Example:

```
import threading

def task():
    for _ in range(1000000):
        pass

t1 = threading.Thread(target=task)
t2 = threading.Thread(target=task)

t1.start()
```

```
t2.start()
```

- ❑ Threads do NOT run truly in parallel (CPU-bound tasks)

Why GIL Exists?

- Simplifies memory management
- Prevents race conditions
- Makes Python easier to implement

❑ Important Distinction:

Task Type	GIL Impact
CPU-bound	Slow (GIL blocks threads)
I/O-bound	Fast (GIL released during I/O)

Backend Insight:

- Use **async** for I/O tasks
- Use **multiprocessing** for CPU-heavy tasks

2.2 Iterators (Core Concept)

❑ What is an Iterator?

An iterator is an object that:

- Returns elements one by one

- Uses `__iter__()` and `__next__()`

Example:

```
nums = [1, 2, 3]

it = iter(nums)

print(next(it)) # 1
print(next(it)) # 2
```

Key Idea:

- Data is accessed sequentially
- No need to store all data at once

Backend Use:

- Streaming data
- Processing large datasets

2.3 Generators (Memory Efficient)

□ What is Generator?

A generator is a **function that yields values one at a time.**

Example:

```
def count(n):  
    for i in range(n):  
        yield i
```

```
gen = count(3)
```

```
print(next(gen)) # 0  
print(next(gen)) # 1
```

Key Difference from Function:

- Uses `yield` instead of `return`
- Maintains state between calls

Memory Advantage:

List:

```
nums = [i for i in range(1000000)]
```

Generator:

```
nums = (i for i in range(1000000))
```

- Generator uses much less memory

Backend Use Cases:

- Streaming APIs
- Reading large files
- Lazy evaluation

2.4 Generator vs Iterator

Feature	Iterator	Generator
Definition	Object	Function
Memory	Depends	Efficient
Creation	Manual	Easy (yield)
State	Manual handling	Automatic

2.5 Memory Management in Python

□ How Python Manages Memory?

Python uses:

- Reference counting
- Garbage collection

Reference Counting:

Each object has a **reference count**

```
a = [1, 2]
b = a
```

- Reference count increases

When count = 0:

- Object is deleted

Garbage Collection:

Handles:

- Cyclic references

```
import gc
gc.collect()
```

Backend Insight:

- Avoid memory leaks
- Clean unused objects
- Use generators for large data

2.6 Memory Optimization Techniques

Use Generators Instead of Lists

```
(i for i in range(1000000))
```

Avoid Unnecessary Copies

```
b = a # reference, not copy
```

Use Built-in Functions

- Faster
- Optimized

Use slots (Advanced)

```
class User:  
    __slots__ = ["name", "age"]
```

- Reduces memory usage

3. Real-World Examples

Example 1: File Streaming

```
def read_file(file):  
    for line in file:  
        yield line
```

Example 2: Large Data Processing

```
def process_data():  
    for i in range(1000000):  
        yield i * 2
```

Example 3: API Pagination

```
def get_pages():  
    for i in range(5):  
        yield f"Page {i}"
```

Example 4: Memory Efficient Loop

```
for num in (i for i in range(1000)):  
    print(num)
```

4. Code Examples (Important Patterns)

Pattern 1: Custom Iterator

```
class Counter:
    def __init__(self, n):
        self.n = n
        self.i = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.i < self.n:
            self.i += 1
            return self.i
        raise StopIteration
```

Pattern 2: Generator Pipeline

```
def square(nums):
    for n in nums:
        yield n * n
```

Pattern 3: Lazy Evaluation

```
nums = (i*i for i in range(10))
```

Pattern 4: Memory Safe Processing

```
def process(stream):  
    for item in stream:  
        yield item
```

5. Common Mistakes

✗ Thinking threads run in parallel (GIL issue)

✗ Using list instead of generator for large data

✗ Not understanding yield vs return

✗ Memory leaks due to references

✗ Not using lazy evaluation

6. Interview Questions with Answers

Q1: What is GIL?

Lock that allows only one thread to execute Python code at a time

Q2: Does GIL affect all programs?

Only CPU-bound programs

Q3: What is iterator?

Object that returns elements one by one

Q4: What is generator?

Function that yields values lazily

Q5: Difference between list and generator?

- List → stores all values
- Generator → generates on demand

Q6: What is yield?

Keyword that returns value and pauses function

Q7: How Python manages memory?

- Reference counting
- Garbage collection

Q8: When to use generator?

- Large data
- Streaming
- Memory optimization

7. Summary (Quick Revision)

- GIL → one thread at a time
- Use:
 - async → I/O tasks
 - multiprocessing → CPU tasks
- Iterator → sequential access
- Generator → memory efficient
- yield → pauses function
- Python memory:
 - reference counting
 - garbage collection

- Optimize using:
 - generators
 - lazy evaluation

Chapter 10: Coding Patterns

(Writing Clean Python, Common Mistakes, Best Practices for Backend)

1. Introduction (Why This Matters in Backend)

Writing code is easy.

Writing **clean, maintainable, scalable backend code** is hard.

In real backend systems:

- Multiple developers work on same codebase
- Code grows over time
- Bugs appear if structure is poor

□ Interviewers don't just check if your code works.

They check:

- Is your code clean?
- Is it readable?
- Is it scalable?
- Does it follow best practices?

Real Backend Impact:

Bad code:

- Hard to debug
- Difficult to scale
- Full of bugs

Clean code:

- Easy to understand
- Easy to maintain
- Easy to extend

□ If you master coding patterns:

- You write **production-level code**
- You impress interviewers
- You become a strong backend developer

2. Core Concepts

2.1 What is Clean Code?

Clean code is code that is:

- Easy to read
- Easy to understand
- Easy to modify

Bad Code:

```
def f(x):  
    return x*2
```

Clean Code:

```
def double_number(number: int) -> int:  
    return number * 2
```

Key Principles:

- Meaningful names
- Simple logic
- Consistent structure

2.2 Naming Conventions (Very Important)

Rules:

- Variables → lowercase_with_underscores
- Functions → lowercase_with_underscores
- Classes → PascalCase

Example:

```
user_name = "Ali"  
  
def calculate_total():  
    pass  
  
class UserService:  
    pass
```

Backend Insight:

- Good naming reduces confusion
- Helps in large teams

2.3 DRY Principle (Don't Repeat Yourself)

Avoid duplicate code.

Bad:

```
print("User created")
print("User created")
```

Good:

```
def log_user_created():
    print("User created")
```

Backend Benefit:

- Easier maintenance
- Single source of truth

2.4 KISS Principle (Keep It Simple)

Bad:

```
def add(a, b):  
    return (a + b) * 1
```

Good:

```
def add(a, b):  
    return a + b
```

Backend Insight:

- Simpler code → fewer bugs

2.5 Separation of Concerns

Divide code into different responsibilities.

Bad:

```
def process():  
    print("Validate")  
    print("Save to DB")  
    print("Send email")
```

Good:

```
def validate():  
    pass  
  
def save():  
    pass  
  
def send_email():  
    pass
```

Backend Structure:

- Controller
- Service
- Repository

2.6 Writing Reusable Functions

Example:

```
def calculate_discount(price: float, percent: float) ->  
float:  
    return price - (price * percent / 100)
```

Backend Benefit:

- Reusable logic
- Cleaner code

2.7 Avoiding Large Functions

Bad:

```
def big_function():  
    # 100+ lines ✕  
    pass
```

Good:

- Break into smaller functions

Rule:

- One function → one responsibility

2.8 Using Built-in Functions (Optimization)

Bad:

```
total = 0
for i in numbers:
    total += i
```

Good:

```
total = sum(numbers)
```

Backend Benefit:

- Faster
- Cleaner

2.9 List & Dict Comprehensions

Bad:

```
result = []
for i in range(10):
    result.append(i*i)
```

Good:

```
result = [i*i for i in range(10)]
```

2.10 Error Handling Best Practices

Bad:

```
try:
    x = 10/0
except:
    pass
```

Good:

```
try:
    x = 10/0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

2.11 Avoid Global Variables

Bad:

```
count = 0
```

Good:

- Use function arguments
- Use classes

2.12 Writing Pythonic Code

Pythonic code means:

- Clean
- Simple
- Readable

Example:

```
if not users:  
    print("Empty")
```

Instead of:

```
if len(users) == 0:
```

2.13 Code Formatting (PEP 8)

Follow Python style guide:

- Proper indentation
- Line length
- Naming

Example:

```
def greet(name: str) -> str:  
    return f"Hello {name}"
```

3. Real-World Examples

Example 1: Clean API Function

```
def get_user(user_id: int) -> dict:  
    return {"id": user_id}
```

Example 2: Service Layer

```
def create_user(data: dict):  
    validate(data)  
    save(data)
```

Example 3: Data Processing

```
numbers = [1, 2, 3]
squares = [n*n for n in numbers]
```

Example 4: Clean Validation

```
def validate_age(age: int):
    if age < 0:
        raise ValueError("Invalid age")
```

4. Code Examples (Important Patterns)

Pattern 1: Guard Clauses

```
def process(data):
    if not data:
        return "Empty"
    return data
```

Pattern 2: Dictionary Mapping

```
operations = {
    "add": lambda a, b: a + b
```

```
}
```

Pattern 3: Early Return

```
def check(x):  
    if x < 0:  
        return "Invalid"  
    return "Valid"
```

Pattern 4: Clean Loop

```
for user in users:  
    print(user)
```

5. Common Mistakes

✗ Writing long functions

✗ Poor variable naming

✗ Repeating code

✗Ignoring edge cases

✗Using global variables

✗Not following PEP 8

✗Overcomplicating logic

6. Interview Questions with Answers

Q1: What is clean code?

Readable, maintainable code

Q2: What is DRY?

Avoid repeating code

Q3: What is KISS?

Keep code simple

Q4: Why avoid global variables?

Hard to manage and debug

Q5: What is Pythonic code?

Code that follows Python best practices

Q6: Why use built-in functions?

Optimized and readable

Q7: What is separation of concerns?

Divide code into responsibilities

Q8: How to improve code readability?

- Naming
- Formatting
- Simplicity

7. Summary (Quick Revision)

- Write clean, readable code

- Follow:
 - DRY
 - KISS
 - PEP 8
- Use:
 - meaningful names
 - small functions
 - built-in methods
- Avoid:
 - global variables
 - long functions
 - duplicate code
- Structure backend code properly

☐ Master coding patterns =

✓ Production-quality backend code

✓ Better teamwork

✓ Strong interview performance ☐